AWS + Node.js L1 Interview Cheatsheet (Definitions • ELI5 • Reasons • Snippets • Twisters)

Prepared for: LTI Mindtree L1 (AWS Node Dev) — concise, practical, interview-ready

API Gateway — HTTP API vs REST API (AWS product types)

Definition (for interview): Both are API Gateway products. HTTP API is cheaper, lighter. REST API adds managed features: API keys + usage plans (per■client throttling), request validation with models, AWS WAF integration, and private endpoints. Pick REST when you need these extras; otherwise choose HTTP API to save cost/latency.

ELI5: Two front doors. The fancy door (REST) has guards and gadgets built■in. The simple door (HTTP) is cheaper but you must do more yourself.

Reason you can't "just do it" on HTTP API: You *can* read a Bearer token in your Lambda, but HTTP API does not provide **usage plans / API keys / request validation / WAF / private endpoints** as managed features. You'd have to rebuild those yourself, which defeats the point when you need them.

Twister: "If I put a custom API token in a header on HTTP API, can I rate■limit per client?" → Only if you build storage, counters, and throttling logic yourself. REST API usage plans do this for you.

```
Example idea (REST Usage Plan + API Key — pseudocode steps)
1) Create REST API method, mark "API Key Required"
2) Create a Usage Plan with throttle/burst
3) Create an API Key and add it to the plan
4) Deploy stage, associate stage with usage plan
5) Call with: x-api-key: <key> (throttling now per key)
```

AWS Lambda — Provisioned Concurrency (PC)

Definition: Pre■initializes N execution environments so first requests are served with warm start latency (double■digit ms). Great for user■facing, spiky traffic.

ELI5: Hotel rooms pre■warmed. Guests skip check■in lines.

What it fixes: Cold starts at the first hit. You still optimize init time (move SDK clients outside handler), but PC makes the first call fast as well.

Twister: "If traffic spikes 10x above PC, are all calls fast?" → Only up to the PC level; extra concurrency uses on■demand environments (can cold start).

```
PC quick sketch (AWS CLI)
aws lambda put-provisioned-concurrency-config   --function-name myFn --qualifier 1 --provisioned-concur
```

Idempotency (avoid double charges)

Definition: Client sends a unique Idempotency-Key; server stores the first result and returns the same result on retries.

ELI5: If the cashier sees the same receipt number again, they don't charge twice.

Why a different key is a new order: Server treats different keys as new; therefore also dedupe by a business key

(e.g., orderId+userId) or hash of the payload for a short window.

Twister: “Client changed the Idempotency-Key accidentally—are we safe?” → Yes, if you additionally enforce uniqueness on domain identifiers (e.g., orderId unique), so the second attempt fails cleanly or returns existing result.

```
Node/Express sketch (in■memory example; use Redis/Dynamo in prod)
const store = new Map();
app.post('/charge', async (req, res) => {
  const key = req.get('Idempotency-Key');
  const orderId = req.body.orderId;
  if (!key || !orderId) return res.status(400).json({error:'missing keys'});

  if (store.has(key)) return res.json(store.get(key)); // same idempotency key

  // ALSO guard by orderId uniqueness
  if ([...store.values()].some(v => v.orderId === orderId)) {
    return res.status(409).json({error:'duplicate orderId'});
  }
  const result = { ok:true, orderId, chargeId:'ch_123' };
  store.set(key, { ...result, orderId });
  res.json(result);
});
```

AWS X■Ray — Express quick setup

Definition: Tracing SDK that captures request segments and downstream subsegments (AWS SDK and HTTP calls), viewable in X■Ray Service Map.

ELI5: A delivery map that shows where time is spent along the route.

Steps: install sdk, wrap express with openSegment/closeSegment, capture AWS SDK + HTTPs globally, add custom annotations when needed.

```
Minimal Express integration
const AWSXRay = require('aws-xray-sdk');
const express = require('express');
const app = express();
app.use(AWSXRay.express.openSegment('MyApp'));
app.get('/ok', (req,res)=>res.send('ok'));
app.use(AWSXRay.express.closeSegment());
app.listen(3000);
```

Amazon S3 — Pre■signed URLs

Definition: Time■limited, single■object URLs that let browsers upload/download directly to S3 without exposing AWS credentials.

ELI5: A one■time, time■boxed ticket for a specific file.

Common inputs: Bucket, Key, HTTP method (PUT/GET), ContentType, Expires.

```
AWS SDK v3 (Node) — getSignedUrl
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
import { getSignedUrl } from '@aws-sdk/s3-request-presigner';
const s3 = new S3Client({ region: 'ap-south-1' });
const url = await getSignedUrl(s3,
  new PutObjectCommand({ Bucket:'my-bkt', Key:'avatar.png', ContentType:'image/png'}),
  { expiresIn: 3600 });
```

```
// PUT file with curl or fetch using this URL
```

Eventing — SQS vs SNS vs EventBridge

ELI5: SQS = to■do queue workers pull; SNS = megaphone push to many; EventBridge = smart post office with rules/filters and integrations.

When to choose: backlog/worker control → SQS; fan■out notifications → SNS; routing/filtering/cross■account/SaaS targets → EventBridge.

Networking — Security Group vs NACL

SG: instance/ENI■level, **stateful** (allow reply traffic automatically), allow rules only.
NACL: subnet■level, **stateless** (must allow both directions), ordered allow/deny.

ELI5: SG = bouncer at the door; NACL = gate at the neighborhood.

Databases — RDS Multi■AZ vs Read Replica

Multi■AZ: high availability/failover (synchronous standby). Not for read scaling.
Read Replica: read scaling/reporting (asynchronous). Not automatic failover.

DynamoDB — GSI vs LSI

GSI: different partition/sort keys; add anytime; spans all partitions; separate throughput.
LSI: same partition key, alternate sort key; must create with table; item■collection size limit applies.

MongoDB — Sharding

Definition: Horizontal partitioning across shards using a shard key; mongos routes queries; config servers hold metadata; balancer moves chunks to keep balance.

ELI5: Split a huge phone book into many shelves; a librarian (mongos) sends you to the right shelf.

Distributed Systems — CAP Theorem

Definition: In the presence of network partitions, a distributed system must choose between Consistency (every read sees latest write) and Availability (every request gets a response). Partition tolerance is required in real systems. Hence, you pick CP or AP during partitions.

Node.js Runtime — Single thread, Cluster vs Worker Threads

Event loop runs your JS on one thread. For multi■core HTTP, use **cluster** (many Node processes sharing one port). For CPU■heavy JS, use **worker_threads** (parallel threads inside one process; can share memory).

```
Cluster tiny demo
const cluster = require('node:cluster');
const http = require('node:http');
const os = require('node:os');
if (cluster.isPrimary) { for (let i=0;i<Math.min(os.cpus().length,4);i++) cluster.fork(); }
else { http.createServer((req,res)=>res.end('pid='+process.pid)).listen(3000); }
```

```
Worker Threads tiny demo
const { isMainThread, Worker, parentPort, workerData, threadId } = require('node:worker_threads');
if (isMainThread) { new Worker(__filename,{workerData:{n:35}}); }
```

```
else { const fib=n=>n<2?n:fib(n-1)+fib(n-2); parentPort.postMessage({threadId,res:fib(workerData.n)});
```

Node — Streams & pipeline()

Use streams to process large data without loading it all into memory; pipeline() connects streams and forwards errors properly (backpressure aware).

Express — Essentials

• Error handler signature is four args: (err, req, res, next) — register after routes.
• Helmet: add safe security headers quickly (`app.use(helmet())`).
• CORS: explicitly allow your frontend origin.
• Rate limiting: throttle abuse with express-rate-limit.

```
Express error handler
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({ error: 'Something broke' });
});
```

Redux Toolkit — default middleware

By default configureStore adds: redux■thunk (both envs). In development it also adds the immutability check and serializability check middleware (can be turned off or customized via getDefaultMiddleware).

JavaScript — Promise combinators

• all: wait for all; fail fast on first reject.
• allSettled: never throws; report of each promise outcome.
• race: settle on the very first result (resolve or reject).
• any: resolve on first fulfillment; reject only if all reject (AggregateError).

JavaScript — WeakMap & WeakSet

Attach metadata to objects without preventing GC. Keys (WeakMap) / values (WeakSet) must be objects; entries can disappear when objects are collected; not iterable; no size().

React — createPortal, useLayoutEffect, useContext, useReducer

• createPortal: render children into a different DOM node (modals/tooltips).
• useLayoutEffect: read/measure layout and update before paint (avoid flicker); prefer useEffect unless measuring.
• useContext: read shared values without prop drilling.
• useReducer: group complex updates in one reducer; dispatch actions.

Common Twisters

1) setTimeout ordering:
setTimeout(()=>1,2000); console.log(2); setTimeout(()=>3,0); console.log(4)
→ Output: 2, 4, 3, 1 (timers queue runs after sync code; 0ms still queues).
2) Arrow vs regular method `this`:
a: ()=>console.log(this) → lexical `this` (likely global/undefined)
b: function(){console.log(this)} → bound to caller (`obj`).

References

- AWS API Gateway HTTP vs REST comparison – docs.aws.amazon.com/apigateway (HTTP API vs REST API)
- AWS Lambda Provisioned Concurrency – docs.aws.amazon.com/lambda
- AWS X■Ray SDK for Node.js + Express middleware – docs.aws.amazon.com/xray
- AWS S3 pre■signed URLs (SDK v3) – docs.aws.amazon.com/AWSJavaScriptSDK/v3, AWS blog
- AWS decision guide: SQS vs SNS vs EventBridge – docs.aws.amazon.com/decision-guides
- AWS VPC: Security Groups (stateful) vs NACLs (stateless) – docs.aws.amazon.com/vpc
- RDS Multi■AZ vs Read Replicas – aws.amazon.com/rds/features/read-replicas, RDS docs
- DynamoDB GSI vs LSI – docs.aws.amazon.com/amazondynamodb
- MongoDB Sharding – mongodb.com/docs/manual/sharding
- CAP Theorem – en.wikipedia.org/wiki/CAP_theorem, IBM Think article
- Node docs: cluster, worker_threads, stream.pipeline – nodejs.org/api
- Express error handling docs – expressjs.com
- Helmet docs – helmetjs.github.io
- Redux Toolkit getDefaultMiddleware – redux-toolkit.js.org
- MDN: Promise.all / allSettled / any / race; WeakMap / WeakSet – developer.mozilla.org
- React docs: createPortal, useLayoutEffect, useContext, useReducer – react.dev